FORWARD CHAINING

Aim:

To implement the Forward Chaining algorithm in Prolog.

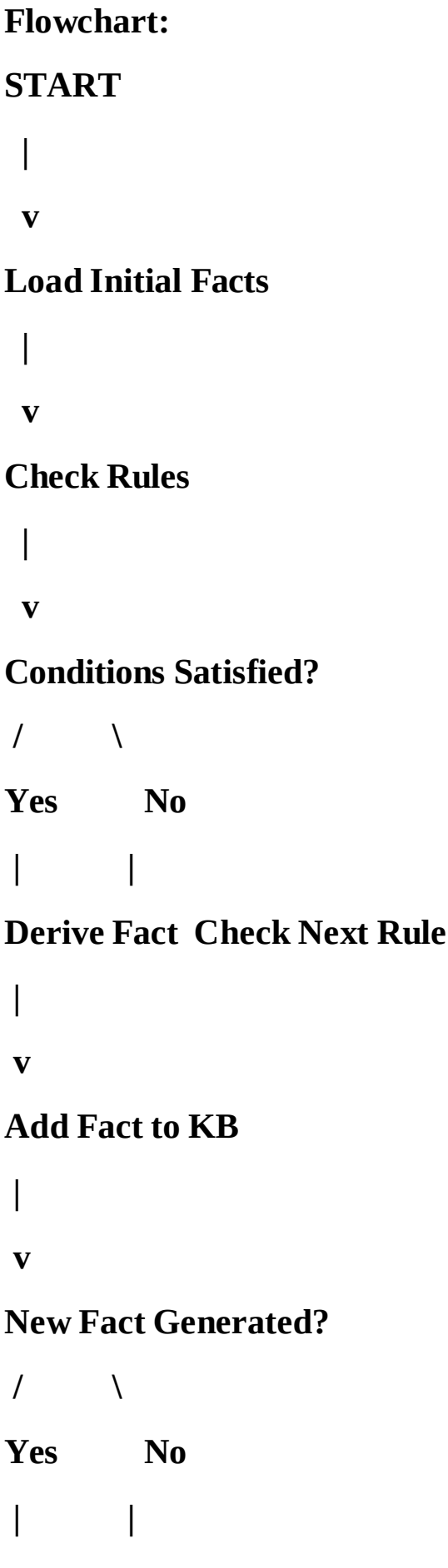
Objective:

To derive new facts from an initial set of facts by repeatedly applying rules.

Algorithm:

1. **Start.**
2. **Define initial facts.**
3. **Define rules.**
4. **Check whether all conditions of a rule are satisfied.**
5. **If satisfied, derive the conclusion.**
6. **Add the new fact to the knowledge base.**
7. **Repeat until no new facts can be derived.**
8. **Stop.**

Flowchart:


START
Load Initial Facts
Check Rules
Conditions Satisfied?
Yes
No
Derive Fact
Check Next Rule
Add Fact to KB
New Fact Generated?
Yes
No

Repeat STOP

Prolog Program:

```
% Forward Chaining

fact(fever).
fact(cough).

rule([fever, cough], flu).
rule([flu], doctor_visit).
rule([doctor_visit], treatment).

forward :-
    collect_facts(Facts),
    forward_chain(Facts, FinalFacts),
    write('Final Facts: '),
    write(FinalFacts),
    nl.

collect_facts(Facts) :-
    findall(Fact, fact(Fact), Facts).

forward_chain(Facts, FinalFacts) :-
    findall(
        Conclusion,
```

```
        (
            rule(Conditions, Conclusion),
            all_present(Conditions, Facts),
            \+ member(Conclusion, Facts)
        ),
        NewFacts
    ),
    (
        NewFacts = [] ->
        FinalFacts = Facts
        ;
        append(Facts, NewFacts, UpdatedFacts),
        forward_chain(UpdatedFacts, FinalFacts)
    ).

all_present([], _).

all_present([H|T], Facts) :-
    member(H, Facts),
    all_present(T, Facts).
```

Query

```
?- forward.
```

Output

```
Final Facts: [fever,cough,flu,doctor_visit,treatment]
```

true.

Result:

Thus, Forward Chaining is successfully implemented to derive new facts from the knowledge base.